CSE107 — Lab #9

Binary File Handling and the Ceasar Encryption

Lab Date: 05/12/2025 — Due: 14/12/2025 11:59 pm

Student: Name Surname
Student ID: 240104004006
Section: Section 2
Instructor: İlhan Aytutuldu

1. Objectives

We practiced Binary File Handling and the Caesar Cipher in C. We defined the student struct (record), encrypted text fields (name, department), and used fseek(), fread(), and fwrite() for secure and efficient record storage.

2. Environment

Fill in the following ones you used in the lab and **give brief information** for each one of them.

- **Operating System: Linux**
- **IDE:** VSCode / GCC

3. Problem Solving

3.1 The main problem was the input buffer issue after scanf(). The solution was adding getchar() after scanf() to properly read all user inputs.

3.2 Binary files are faster and use less space than text files. They allow Random Access (fseek()) because all student records have a fixed size.

3.3 Numbers are saved as raw bytes for accuracy and to keep their size fixed.

3.4 Using variable-length strings breaks fseek() because the record size changes. The solution was using fixed-size arrays (char name[100]) to keep the struct size constant.

3.5
the_create(): Uses "ab" mode and fwrite() to encrypt and add a new record to the end.
the_get_a_record(): Uses fseek() to jump, fread() to read, and then decrypts the record.
the_get_all_records(): Loops with fread() to read and decrypt all records sequentially.

3.6
fopen(): Opens the file. It needs the file name (FILENAME) and the mode ("ab" or rb"). It returns a FILE* pointer.

fseek(): Moves the file pointer. It lets us jump to a specific record without reading the whole file. We use it to calculate the exact byte position for the record we want.
fwrite(): Writes data to the file. It takes the student struct (&s) and the size (sizeof(student)) and writes the raw bytes to the file. Used in the_create().
fread(): Reads data from the file. It reads the raw bytes into the student struct (&s). Used in the_get_a_record() and the_get_all_records().
fclose(): Closes the file. This must be done after finishing work to save the data permanently.

3.7 If we give fseek() a wrong byte offset, the file pointer goes to an unintended place. This causes problems when we try to read the data. If the pointer lands in the middle of a record, fread() will read a mix of two different records, resulting in corrupted and meaningless output. If the offset is too large and goes past the end of the file, fread() will simply fail.

3.8 If a record number doesn't exist, fseek() moves the pointer past the end, causing fread() to fail (return 0). The program handles this by printing: "no record at this order!"

3.9 File mode selection is important because it sets the permissions (read or write). A wrong mode, like using "wb" by accident, will delete all existing data, or "rb" will cause writing to fail.

3.10 If the file is not closed, the latest data might be lost (stuck in memory buffer), and the system's ability to open other files later will be harmed (resource leak).

3.11 A structure (struct) in C is a way to group different types of data (like name, ID, and grade) into one single unit. They are used to model real-world items, such as our student record, which makes the code clear. They are crucial for binary file handling because the struct defines the exact layout and size of the record that fread() and fwrite() use.

3.12 The sizeof operator tells you the size in bytes of a variable or type. We use this size to correctly calculate the jump distance for fseek() and to tell fread() and fwrite() the exact amount of data to move. This ensures we read and write one complete record correctly every time.

3.13 The Caesar Cipher is a simple code that shifts every letter forward by a fixed key, which is 12 in our code. The process checks if a character is a letter, calculates its index, adds 12, and uses the modulo (%26) to ensure the shift wraps around the alphabet. Non-letters are ignored.

3.14 We add 26 during decryption because C's modulo operator (%) can give a negative result when we subtract the key (index - 12) if the index is small. Adding 26 guarantees the number is positive before the modulo, which correctly implements the backward wrap-around (e.g., shifting 'A' back 1 gives 'Z's index).

3.15 If you give the encryption function any character that is not a letter (like a space or a number), the output will be the same character, unchanged. This happens because the function skips the encryption code and directly executes the return ch; statement.

4. Labwork Evidence — Screenshots

The lab work involved four main steps: First, we compiled the C code successfully in the terminal . Second, we ran the program and selected option 1 the_create to add a new record, where the input data was automatically encrypted before being saved to the binary file. Third, we selected option 2 the_get_a_record to demonstrate random access using fseek(), which allowed us to jump directly to a specific record, read it, and decrypt the data for display. Finally, we selected option 3 to list all records sequentially, confirming all data could be successfully read and decrypted.

```
ztoptas@MSI:/mnt/c/Users/ms1/OneDrive/Desktop/CSE101/labworks$ cc labwork9.c
ztoptas@MSI:/mnt/c/Users/ms1/OneDrive/Desktop/CSE101/labworks$ ./a.out

========== menu ==========
1. add a record
2. get a record
3. get all records
0. exit
enter your choice: 1
name of the student: özge
id: 240104004006
department: cse
grade: 90
record added successfully!

========== menu ==========
1. add a record
2. get a record
3. get all records
0. exit
enter your choice: 2
enter the order of the record (starting from 1): 1

=== record (order 1) ===
| name       | özge
| id         | 240104004006
| department | oqzs
| grade      | 90
```

```
========== menu ==========
1. add a record
2. get a record
3. get all records
0. exit
enter your choice: 3

=== all records ===

| record     | 1
| name       | özge
| id         | 240104004006
| department | ceng
| grade      | 90

| record     | 2
| name       | begüm
| id         | 123
| department | ceng
| grade      | 100

| record     | 3
| name       | özge
| id         | 240104004006
| department | cse
| grade      | 90

========== menu ==========
1. add a record
2. get a record
3. get all records
0. exit
enter your choice: 0
exiting...
ztoptas@MSI:/mnt/c/Users/ms1/OneDrive/Desktop/CSE101/labworks$
```